

Warm up

What does the following code attempt to do? Should we call `increment1` or `increment2` to achieve the intended behaviour? Perhaps neither?

```
void increment1(void* x) {
    *x += 1;
}

void increment2(void* x) {
    *(int*)x += 1;
}

int main(void) {
    unsigned char a = 255;
    void* p = &a;
    incrementX(p); /* either increment1 or increment2 */
    printf("a=%d\n", a);
    return 0;
}
```

Warm up

```
void increment1(void* x) { *x += 1; }
```

- `increment1` does not compile because we cannot dereference a `void*` pointer (last precept)

```
void increment2(void* x) { *(int*)x += 1; }
```

```
int main(void) {  
    unsigned char a = 255;  
    void* p = &a;  
    incrementX(p); /* either increment1 or increment2 */  
    printf("a=%d\n", a);  
    return 0;  
}
```

- `increment2` compiles, but exhibits undefined behaviour
 - `a` is an `unsigned char`, but we are dereferencing it as an `int`
 - Violates the strict aliasing rule: an object in memory must be accessed only through a pointer of a compatible type
 - ◇ Allows the compiler to make certain optimisations

Precept 12: Function pointers

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

March 17, 2026

Overview

- 1 Function pointers
- 2 Writing a higher-order function: `map`
- 3 Summary

Function pointer syntax: declaration

- A function pointer is a variable that stores the address of a function
 - Like a pointer, but points to the text segment, where the compiled code is stored

- Declaring a function pointer:

```
return_type (*pf)(param1_type, param2_type, ...);
```

- Example: pointer to function that takes two integers and returns an integer: `int (*pf)(int, int);`
- Note: the parentheses around `*pf` are necessary to indicate that `pf` is a pointer to a function
 - `int *pf(int, int);` declares a function that takes two integers and returns an `int*`

Function pointer syntax: assignment and usage

- Assigning a function to a function pointer:
 - `pf = &func;`
 - `pf = func;`
 - ◇ `func` decays to a pointer to the function
 - `pf = *func;`
 - ◇ Quite unusual, but valid C syntax
 - ◇ Works because `func == &func  $\iff$  *func == func`
- Calling a function through a function pointer:
 - `result = (*pf)(arg1, arg2, ...);`
 - `result = pf(arg1, arg2, ...);`
- Any combination of the above can be used for assignment and calling
 - Common idiom: `pf = func;` and `result = pf(arg1, arg2);`

Why bother with function pointers?

- Function pointers allow us to write more flexible and reusable code
- They enable *higher-order programming*, where functions can be passed as arguments to other functions or returned as values
- Common use cases:
 - *Callbacks*: pass a function to be called later (e.g., `qsort` takes a comparator)
 - *Dynamic dispatch*: choose which function to call at runtime based on program state
 - *Polymorphism*: store function pointers in structs to simulate object-oriented behaviour

How does qsort use function pointers?

- qsort is a standard library (stdlib.h) function that sorts an array

```
void qsort(void *base, size_t nel, size_t width,  
          int (*compar)(const void *, const void *));
```
- It takes a pointer to the array (base), the number of elements (nel), the size of each element (width)
- How does it know *how* to sort generic elements?
 - compar function pointer allows the caller to specify how to compare two elements
 - Ordering is determined by return value of compar: negative if the first argument should come before the second, zero if equal, positive if after (similar to strcmp)
- compar must point to a function that takes two `const void*` arguments (pointers to elements being compared) and returns an `int`
- Can omit parameter names in declaration:

```
void qsort(void *, size_t, size_t, int (*)(const void *, const void *));
```


Example: using qsort with function pointers

```
static int cmpDouble(const void *pv1, const void *pv2) {
    if (*(double*)pv1 < *(double*)pv2) return -1;
    if (*(double*)pv1 > *(double*)pv2) return 1;
    return 0;
}

int main(void) {
    /* ... set up array of numElements doubles ... */
    qsort(array, numElements, sizeof(double), cmpDouble);
}
```

- Do we need to cast pv1 and pv2 to `double*` before dereferencing?
 - Yes, because pv1 and pv2 are `void` pointers, which cannot be dereferenced directly

Can cmpDouble take double* parameters instead?

```
static int cmpDouble(const double *pd1, const double *pd2) {  
    if (*pd1 < *pd2) return -1;  
    if (*pd1 > *pd2) return 1;  
    return 0;  
}
```

- Avoid having to cast the arguments before dereferencing!
- But, recall: qsort expects a function pointer to a function that takes two `const void*` parameters
- Calling qsort directly with cmpDouble above produces a type error

```
error: incompatible function pointer types passing  
'int (const double *, const double *)' to parameter of type  
'int (* _Nonnull)(const void *, const void *)'  
[-Wincompatible-function-pointer-types]
```

- Solution: type-cast the function pointer!

```
qsort(array, numElements, sizeof(double),  
      (int (*)(const void*, const void*))cmpDouble);
```

Overview

- 1 Function pointers
- 2 Writing a higher-order function: `map`
- 3 Summary

map

- Sometimes, we want to apply the same operation to every element in a data structure (e.g., an array)
 - Increment every element by one
 - Convert an array of temperatures from Celsius to Fahrenheit
 - Print every element to the console
 - Compute the sum of all elements
 - etc.
- One approach: write a separate function for each operation
- Limitation: How do we anticipate all possible operations the user might want to perform?
- Better approach: write a single function that takes a function pointer as an argument, and applies that function to each element → `map`!
- Other higher-order functions: `filter` (keep only elements that pass a filter), `reduce` (combine all elements), etc.

Stack_map

```
/* Apply function *pfApply to each element of oStack,
   passing pvExtra as an extra argument. That is, for
   each element pvItem of oStack, call
   (*pfApply)(pvItem, pvExtra). */
void Stack_map(Stack_T oStack,
               void (*pfApply)(void *pvItem, void *pvExtra),
               const void *pvExtra);
```

- pfApply is a function pointer to a function that takes two `void*` parameters and returns `void`
 - Client provided, can do whatever the client wants
- Why do we need pvExtra?
 - Sometimes, the operation we want to perform on each element requires extra information (e.g., the increment value for incrementing each element, or the format string for printing each element)
 - The extra data provides *closure* (environment / context) to the callback function without using global variables

Let's implement Stack_map

Stack_map should iterate over each element in the stack and call pfApply on it, passing pvExtra as the second argument:

```
void Stack_map(Stack_T oStack,
               void (*pfApply)(void *pvItem, void *pvExtra),
               const void *pvExtra) {
    struct StackNode *psCurrentNode;

    assert(oStack != NULL);
    assert(pfApply != NULL);

    for (psCurrentNode = oStack->psFirstNode;
         psCurrentNode != NULL;
         psCurrentNode = psCurrentNode->psNextNode) {
        (*pfApply)((void*)psCurrentNode->pvItem,
                   (void*)pvExtra);
    }
}
```

- Should we also assert if pvExtra is not NULL?
 - No, some operations may not need extra data (e.g., freeing each elt)

Using Stack_map to print all elements

```
/* Print string pvItem using format string pvExtra. */
static void printString(void *pvItem, void *pvExtra) {
    assert(pvItem != NULL);
    assert(pvExtra != NULL);
    printf((char*)pvExtra, (char*)pvItem);
}

int main(void) {
    Stack_T oStack1 = Stack_new();
    /* ... push some strings onto oStack1 ... */
    Stack_map(oStack1, printString, "%s\n");
    /* ... cleanup ... */
    return 0;
}
```

Using Stack_map to sum all elements

```
/* Accumulate double *pvItem into *pvExtra. */
static void sumDouble(void *pvItem, void *pvExtra) {
    double *pdItem;
    double *pdSum;
    assert(pvItem != NULL);
    assert(pvExtra != NULL);
    pdItem = (double*)pvItem;
    pdSum = (double*)pvExtra;
    *pdSum += *pdItem;
}

int main(void) {
    Stack_T oStack2 = Stack_new();
    /* ... push some doubles onto oStack2 ... */
    double dSum = 0.0;
    Stack_map(oStack2, sumDouble, &dSum);
    printf("The sum is %g.\n", dSum);
    /* ... cleanup ... */
    return 0;
}
```


Using Stack_map to free all elements

```
/* Free pvItem. pvExtra is unused. */
static void freeDouble(void *pvItem, void *pvExtra) {
    assert(pvItem != NULL);
    free(pvItem);
}

int main(void) {
    Stack_T oStack3 = Stack_new();
    /* ... push heap-allocated doubles onto oStack3 ... */
    Stack_map(oStack3, freeDouble, NULL);
    Stack_free(oStack3);
    return 0;
}
```

- Observe: we pass NULL for pvExtra since freeDouble does not require extra data

Overview

- 1 Function pointers
- 2 Writing a higher-order function: `map`
- 3 Summary

Summary

- How to declare, assign and use function pointers
- Function pointers enable higher-order programming, allowing us to write more flexible and reusable code
- Implementing a `map` function in our stack ADT
- Dates to remember:
 - Assignment 3: due Tue, 3/24 at 9pm
 - Assignment 3 Quiz: Wed, 3/25 in lecture
- Acknowledgements
 - The stack implementations in these slides are adapted from the precept materials, written by Prof. Bob Dondero
 - Can be accessed at <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w08p1/stack6/>